NexusOS

AI Business Operating System

Part 03.0

Linear Issue WAL-10

Database Modeling

Project	NexusOS — AI Business Operating System
Part	Part 03.0
Linear issue	WAL-10
Report title	Database Modeling
Generated	2026-07-19
Purpose	Standalone Linear attachment (manual upload)
Source report(s)	docs/reports/Part-03.0/Part-03.0-Database-Modeling.md

This PDF is a presentation export of repository engineering report content. Source report files in the repository were not modified.

Evidence Statement

This report is a retrospective engineering reconstruction produced after implementation. Its contents are derived exclusively from repository evidence. It is not a contemporaneous engineering document. No engineering events are fabricated. Where repository evidence is unavailable, that limitation is explicitly documented.

Table of Contents

Part 03.0 — Database Modeling

- Evidence Statement
- Objective
- Model reconstructed from repository evidence
  - Shared column set (`TenantBusinessMixin`)
  - Key modeling rules evidenced
  - Entity inventory (initial migration tables)
- Repository evidence
- Outcome
- Evidence limitations (shared)
  - Phase-specific limitation

Report Content

Content below is copied from the source repository report(s) without alteration.

Part 03.0 — Database Modeling

Linear: WAL-10 **Reconstruction date:** 2026-07-19

Evidence Statement

This report is a retrospective engineering reconstruction produced after implementation.

Its contents are derived exclusively from repository evidence.

It is not a contemporaneous engineering document.

No engineering events are fabricated.

Where repository evidence is unavailable, that limitation is explicitly documented.

Objective

Produce a logical/physical model: entities, attributes, keys, relationships, normalization, and tenant-aware structure ready for ORM/DDL implementation.

Model reconstructed from repository evidence

Shared column set (`TenantBusinessMixin`)

`id` (UUID v7) · `company_id` · `created_at` · `updated_at` · `created_by` · `updated_by` · `deleted_at` · `is_active`

Key modeling rules evidenced

- Per-tenant uniqueness: `(company_id, email)` on users; SKU; invoice_number; role name; employee_number; attendance `(employee_id, date)`; inventory `(warehouse_id, product_id)`; global `companies.slug`
- History tables omit soft-delete/active mutation fields
- Money columns use `NUMERIC(14,2)`
- Statuses use VARCHAR enums (`native_enum=False`)
- `created_by`/`updated_by` use `ON DELETE SET NULL` with `use_alter=True` (later Part 03.1 found some of these FKs missing in live DB — see limitations)

Entity inventory (initial migration tables)

37 tables created in `0a961c2ea2cf`, including companies, permissions, roles, users, association tables, and domain tables listed in the Schema Implementation report.

Repository evidence

- `docs/product/database.md` (entity map, uniqueness, indexes)
- `backend/app/models/*.py`
- `backend/migrations/versions/20260709_0a961c2ea2cf_initial_schema.py`
- Commit `838db64`

Outcome

Complete normalized, tenant-aware model expressed in SQLAlchemy and Alembic.

Evidence limitations (shared)

- Part 03.0 code primarily landed in git commit `838db64` (2026-07-09) under the commit message “chore: validate and stabilize backend foundation”, bundled with broader foundation scaffolding. There is no separate contemporaneous commit uniquely titled “Part 03.0”.
- Linear issues WAL-7…WAL-17 were created on 2026-07-18 as a historical execution hierarchy after the schema already existed in the repository.
- No contemporaneous Part-03.0 independent-review finding IDs, meeting notes, or signed review PDF exist in the repository (unlike Part 03.1’s report pack).
- Structural integrity remediations (composite cross-tenant FKs, soft-delete-safe unique indexes, missing `created_by`/`updated_by` FKs) are **Part 03.1** evidence (`f521d38`, revisions `3a2aec52f6d2` / `217e90d00cc0`) and are not attributed as silent Part 03.0 remediations.

Phase-specific limitation

No separate ERD artifact (Visio/dbdiagram export) exists beyond the mermaid diagram in `database.md` and the live models/migration.